College Management System

Feature Implementation Guide

No New Entities • No Breaking Changes • Cursor-Ready Prompts

Overview

This document provides the exact Cursor AI prompts and file-level edit specifications to implement 10 high-value features in your existing Spring Boot CMS backend. Every feature reuses existing entities, repositories, and patterns already present in the codebase. No new database tables are required.

Feature	Tier	Files Touched	Effort
Attendance Defaulter Report	Tier 1	AttendanceRepository, AttendanceService, AttendanceController	~1 hr
Bulk Attendance Marking	Tier 1	AttendanceController, AttendanceService (bulk DTO)	~1 hr
Student Self-Service /me endpoints	Tier 1	StudentController, FeesController, ResultsController, AttendanceController	~1.5 hr
Faculty Self-Service /me endpoints	Tier 1	FacultyController, SubjectController, AttendanceController	~1 hr
Fee Defaulters / Overdue	Tier 2	FeesRepository, FeesService, FeesController	~45 min
Faculty Exam & Result Ownership	Tier 2	ExamService, ResultsService (mirror attendance pattern)	~1.5 hr
Book Return + Fine Calculation	Tier 2	BookIssueService, BookIssueController	~1 hr
Dashboard Stats Endpoint	Tier 2	New DashboardController + DashboardStatsDTO (no entity)	~1 hr

Feature	Tier	Files Touched	Effort
Result Summary per Student	Tier 2	ResultsRepository, ResultsService, ResultsController	~45 min
Upcoming Events / Exams	Tier 2	EventsRepository, ExamRepository + controllers	~30 min

Cursor Master Prompt (Paste This First)

> *Paste this block at the START of every Cursor session before giving individual feature prompts. It sets the ground rules.*

--- CURSOR CONTEXT BLOCK (paste before each feature prompt) ---

You are working on a Spring Boot 3 / Java 21 College Management System. Key rules:

- Package root: com.ayush.College_Management_System
- Security: JWT auth via JwtAuthFilter. SecurityUserAccessor.getCurrentUser() returns AppUserDetails with userId, role (ADMIN/FACULTY/STUDENT), linkedId.
- Roles: Use @PreAuthorize("hasRole('ADMIN')"), hasAnyRole, or manual checks via SecurityUserAccessor.
- DTOs: Always create RequestDTO and ResponseDTO. Never expose entities directly.
- Services: Interface + Impl pattern. Add methods to existing interfaces first, then implement in *ServiceImpl.
- Repositories: Extend JpaRepository. Add JPQL @Query methods as needed.
- Exceptions: Throw ResourceNotFoundException for missing entities. GlobalExceptionHandler handles it.
- DO NOT create new entity classes or new database tables.
- DO NOT modify existing endpoint signatures or break existing behavior.
- Only add new methods and new endpoints alongside existing ones.

Feature 1: Attendance Defaulter Report

What It Does

GET /api/attendance/defaulters?threshold=75 returns students whose attendance percentage in any subject falls below the threshold. Optionally filter by departmentId.

Files to Edit

File	Change Type	What to Add
AttendanceRepository.java	Add query	JPQL query grouping attendance by student+subject, returning count of PRESENT vs total
AttendanceService.java	Add method	List<DefaulterResponse> getDefaulters(double threshold, Long departmentId)
AttendanceServiceImpl.java	Implement	Fetch grouped data, calculate %, filter below threshold
AttendanceController.java	Add endpoint	GET /api/attendance/defaulters with @RequestParam

Cursor Prompt

Copy-paste exactly into Cursor:

In the College Management System Spring Boot project, implement the Attendance Defaulter Report feature. Follow these exact steps:

1. In AttendanceRepository.java, add this JPQL query:

```
@Query("SELECT a.student.id, a.subject.id, a.student.name, a.subject.subjectName, " +
       "COUNT(CASE WHEN a.status = 'PRESENT' THEN 1 END) as presentCount, COUNT(a) as totalCount " +
       "FROM Attendance a GROUP BY a.student.id, a.subject.id, a.student.name, a.subject.subjectName")
List<Object[]> findAttendanceSummaryGrouped();
```

2. DefaulterResponse.java already exists in dto/attendance/. Ensure it has fields: studentId (Long), studentName (String), subjectId (Long), subjectName (String), presentCount (long), totalCount (long), percentage (double).

3. In AttendanceService.java interface, add: List<DefaulterResponse> getDefaulters(double threshold, Long departmentId);

4. In AttendanceServiceImpl.java, implement getDefaulters: call findAttendanceSummaryGrouped(), map Object[] to DefaulterResponse, calculate percentage = (presentCount * 100.0) / totalCount, filter where percentage < threshold. If departmentId is not null, also filter by student's department (join through Student entity if needed, or do post-filter if Student has departmentId field).

5. In AttendanceController.java, add endpoint BELOW existing endpoints (do not touch existing ones):

```
@GetMapping("/defaulters")
```

```
public ResponseEntity<List<DefaulterResponse>> getDefaulters(
    @RequestParam(defaultValue = "75") double threshold,
    @RequestParam(required = false) Long departmentId) {
    return ResponseEntity.ok(attendanceService.getDefaulters(threshold,
departmentId));
}
```

> *The DefaulterResponse DTO already exists. Do not recreate it, just verify its fields match.*

{Here's what was implemented.

1. AttendanceRepository

Your snippet used a.student.name and a.subject.subjectName, which are not on your entities (Student has firstName/lastName, Subject has subName). The query was implemented as:

JOIN a.student s JOIN a.subject sub

SUM(CASE WHEN a.status = 'PRESENT' THEN 1 ELSE 0 END) for present count (same idea as COUNT(CASE WHEN …) and works reliably in Hibernate)

Optional department filter:
WHERE (:departmentId IS NULL OR s.department.id = :departmentId)

Method:

List<Object[]> findAttendanceSummaryGrouped(@Param("departmentId") Long departmentId);
Call with departmentId == null for all departments.

2. DefaulterResponse

Updated to:
studentId, studentName, subjectId, subjectName, presentCount, totalCount, percentage (with @NoArgsConstructor / @AllArgsConstructor for Jackson and use in code).

3. AttendanceService

Added:
List<DefaulterResponse> getDefaulters(double threshold, Long departmentId);

4. AttendanceServiceImpl

Calls findAttendanceSummaryGrouped(departmentId)

Maps each Object[] to DefaulterResponse, computes percentage, keeps rows with percentage < threshold

studentName = trimmed firstName + " " + lastName (or "N/A" if empty)

5. AttendanceController

Added:

GET /api/attendance/defaulters?threshold=75&departmentId= (optional)

It is registered before GET /api/attendance/{id} so the path defaulters is not treated as an id (Spring would otherwise try to parse it as Long and return 400).

Existing endpoints and findDefaulters(double threshold) are unchanged. mvn compile succeeds.

Note: SecurityUserAccessor still only has requireCurrentUser(); this feature does not use it. If you want getCurrentUser() as an alias, that can be added in a follow-up.

}

Feature 2: Bulk Attendance Marking

What It Does

POST /api/attendance/bulk lets a FACULTY or ADMIN mark attendance for an entire class in one call instead of one record at a time.

Files to Edit

File	Change Type	What to Add
dto/attendance/	New DTO	BulkAttendanceRequestDTO with subjectId, date, List<AttendanceRecord> records
AttendanceService.java	Add method	List<AttendanceResponseDTO> bulkMark(BulkAttendanceRequestDTO dto)
AttendanceServiceImpl.java	Implement	Loop records, reuse existing create logic, apply faculty ownership check
AttendanceController.java	Add endpoint	POST /api/attendance/bulk with @PreAuthorize ADMIN or FACULTY

Cursor Prompt

In the College Management System Spring Boot project, implement Bulk Attendance Marking. Follow these steps exactly:

1. Create BulkAttendanceRequestDTO.java in dto/attendance/ package:

```
public class BulkAttendanceRequestDTO {
    private Long subjectId;
    private LocalDate date;
    private List<AttendanceRecord> records;
    // inner static class:
    public static class AttendanceRecord {
        private Long studentId;
        private AttendanceStatus status; // reuse existing enum
    }
}
```

2. In AttendanceService.java, add: List<AttendanceResponseDTO> bulkMark(BulkAttendanceRequestDTO dto);

3. In AttendanceServiceImpl.java, implement bulkMark: Get current user from SecurityUserAccessor. Fetch Subject by subjectId (throw ResourceNotFoundException if missing). If role is FACULTY, verify subject.getFaculty().getId() equals currentUser.getLinkedId(), else throw AccessDeniedException. For each record in dto.getRecords(), build an Attendance entity and save. Collect and return AttendanceResponseDTO list. Do NOT call the existing create() method to avoid redundant security checks inside a loop - build and save directly.

4. In AttendanceController.java, add after existing endpoints:

```
@PostMapping("/bulk")
@PreAuthorize("hasAnyRole('ADMIN', 'FACULTY')")
public ResponseEntity<List<AttendanceResponseDTO>> bulkMark(
    @RequestBody BulkAttendanceRequestDTO dto) {
    return ResponseEntity.status(HttpStatus.CREATED)
           .body(attendanceService.bulkMark(dto));
}
```

{Here's what was added and how it behaves.

1. BulkAttendanceRequestDTO (dto/attendance/)

Fields: subjectId, date, records

Nested AttendanceRecord: studentId, status (AttendanceStatus)

Lombok @Data / @NoArgsConstructor / @AllArgsConstructor for JSON binding

2. AttendanceService

List<AttendanceResponseDTO> bulkMark(BulkAttendanceRequestDTO dto);

3. AttendanceServiceImpl.bulkMark

Resolves user with securityUserAccessor.getCurrentUser() (see below).

Loads subject by subjectId → ResourceNotFoundException if missing.

FACULTY: linkedId must match subject.getFaculty().getId() → AccessDeniedException if not (same rules as single create).

ADMIN: allowed.

Any other role → AccessDeniedException.

Does not call create(); builds Attendance, sets relations, save, maps with existing toResponse.

lectureNumber: fixed to 1 for the whole batch (not in your DTO; DB needs it and uniqueness is per student/subject/date/lecture).

semester: student.getCurrentSemester(), else subject.getSemesterNumber(), else 1.

records null/empty → empty list.

Each studentId → ResourceNotFoundException if student missing.

4. AttendanceController

POST /api/attendance/bulk after the delete handler, 201 CREATED, as specified.

5. SecurityUserAccessor

getCurrentUser() added and delegated to requireCurrentUser() so it matches your project rules.

mvn compile passes.

Note: Because the bulk DTO has no lectureNumber, a second bulk for the same subject + date with the same default lecture 1 can hit the unique constraint for students already marked that day. If you need multiple periods per day, extend the DTO later with lectureNumber (and optionally semester) without changing existing endpoints.
}

Feature 3: Student Self-Service /me Endpoints

What It Does

Students can fetch their own profile, fees, results, and attendance summary using their JWT token. No studentId path variable needed - resolved from token's linkedId.

Files to Edit

File	Change Type	What to Add
StudentController.java	Add endpoint	GET /api/students/me
FeesController.java	Add endpoint	GET /api/fees/me
ResultsController.java	Add endpoint	GET /api/results/me
AttendanceController.java	Add endpoint	GET /api/attendance/me (percentage summary)

Cursor Prompt

In the College Management System Spring Boot project, implement Student Self-Service /me endpoints. Follow these steps:

1. In StudentController.java, add:

```
@GetMapping("/me")
@PreAuthorize("hasRole('STUDENT')")
public ResponseEntity<StudentResponseDTO> getMyProfile() {
    Long linkedId = SecurityUserAccessor.getCurrentUser().getLinkedId();
    return ResponseEntity.ok(studentService.getById(linkedId));
}
```

2. In FeesService.java, add method: List<FeesResponseDTO> getByStudentId(Long studentId); and implement it in FeesServiceImpl using FeesRepository with a findByStudentId query (add to FeesRepository if not present).

3. In FeesController.java, add:

```
@GetMapping("/me")
```

```
@PreAuthorize("hasRole('STUDENT')")
public ResponseEntity<List<FeesResponseDTO>> getMyFees() {
    Long linkedId = SecurityUserAccessor.getCurrentUser().getLinkedId();
    return ResponseEntity.ok(feesService.getByStudentId(linkedId));
}
```

4. Apply the same pattern for ResultsController.java - GET /api/results/me returns all results where student.id = linkedId.

5. In AttendanceController.java, add GET /api/attendance/me that returns List<AttendancePercentageResponse> for the logged-in student across all subjects. Reuse any existing percentage logic already in AttendanceService.

> *SecurityUserAccessor.getCurrentUser().getLinkedId() is the key call. Never accept studentId from path/query for /me endpoints.*

{Here's what was implemented.

1. StudentController

Injected SecurityUserAccessor (it's a bean, not static).

GET /api/students/me — @PreAuthorize("hasRole('STUDENT')"), uses getCurrentUser().getLinkedId(), then studentService.getStudentById(linkedId) (your spec said getById; the real method is getStudentById).

Declared before GET /{id} so "me" is not treated as an id.

If linkedId is null → AccessDeniedException (403).

2. FeesRepository & FeesService

List<Fees> findByStudent_Id(Long studentId) (derived query on student.id; no studentId field on the entity).

List<FeesResponseDTO> getByStudentId(Long studentId) in FeesService / FeesServiceImpl: checks student exists, maps with existing toResponse.

3. FeesController

GET /api/fees/me — same STUDENT + linkedId pattern, above GET /{id}.

4. ResultsRepository & ResultsService

List<Results> findByStudent_Id(Long studentId).

List<ResultsResponseDTO> getByStudentId(Long studentId) implemented like fees.

5. ResultsController

GET /api/results/me — same pattern, above GET /{id}.

6. AttendanceController + AttendanceService

GET /api/attendance/me — @PreAuthorize("hasRole('STUDENT')"), linkedId check, returns List<AttendancePercentageResponse>.

AttendanceService.getMySubjectPercentages(Long studentId) — ensures student exists, loads distinct subject ids via new repo query, then getSubjectPercentage(studentId, subjectId) for each.

AttendanceRepository: new findDistinctSubjectIdsByStudentId JPQL only; existing queries unchanged.

Security / behavior

No studentId in path or query for /me.

mvn test completed successfully (exit 0).

Signup reminder: student users need role=STUDENT and linkedId equal to the students.id row.

}

Feature 4: Faculty Self-Service /me Endpoints

Cursor Prompt

In the College Management System Spring Boot project, implement Faculty Self-Service endpoints. Steps:

1. In FacultyController.java, add GET /api/faculty/me annotated @PreAuthorize("hasRole('FACULTY')"). Resolve linkedId from SecurityUserAccessor.getCurrentUser().getLinkedId() and call facultyService.getById(linkedId).

2. In SubjectController.java, add GET /api/subjects/mine annotated @PreAuthorize("hasRole('FACULTY')"). In SubjectService, add List<SubjectResponseDTO> getByFacultyId(Long facultyId). In SubjectRepository, add: List<Subject> findByFacultyId(Long facultyId); (Spring Data JPA will auto-implement if Subject has a Faculty field mapped as faculty). Implement the service method and wire it to the controller.

3. In AttendanceController.java, add GET /api/attendance/mine annotated @PreAuthorize("hasRole('FACULTY')"). Return all attendance records where the subject's faculty id equals linkedId. Add findBySubjectFacultyId(Long facultyId) to AttendanceRepository.

{Here’s what was added.

1. FacultyController

Injected SecurityUserAccessor.

GET /api/faculty/me — @PreAuthorize("hasRole('FACULTY')"), uses getCurrentUser().getLinkedId(), then service.getById(linkedId).

Placed before GET /{id} so "me" is not bound as an id.

AccessDeniedException if linkedId is null.

2. SubjectRepository / SubjectService / SubjectController

Repository: List<Subject> findByFaculty_Id(Long facultyId) — Spring Data path for Subject.faculty.id (there is no facultyId property; findByFacultyId would not resolve correctly).

Service: List<SubjectResponseDTO> getByFacultyId(Long facultyId) — checks faculty exists (ResourceNotFoundException if not), then maps with existing mapToResponse.

GET /api/subjects/mine — @PreAuthorize("hasRole('FACULTY')"), linkedId check, subjectService.getByFacultyId(linkedId).

Declared before GET /{id}.

3. AttendanceRepository / AttendanceService / AttendanceController

New JPQL (existing queries unchanged): findBySubjectFacultyId(@Param("facultyId") Long facultyId) — WHERE a.subject.faculty.id = :facultyId.

Service: List<AttendanceResponseDTO> getBySubjectFacultyId(Long facultyId) — uses toResponse for each row.

GET /api/attendance/mine — @PreAuthorize("hasRole('FACULTY')"), linkedId check, returns List<AttendanceResponseDTO>.

Placed after student /me, before /{id}.

mvn test completed with exit code 0.

Faculty accounts need linkedId = faculty row id (same as for attendance marking).

}

Feature 5: Fee Defaulters / Overdue

Cursor Prompt

In the College Management System Spring Boot project, implement Fee Defaulters feature. Steps:

1. In FeesRepository.java, add:

```
List<Fees> findByPaymentStatusNot(PaymentStatus status);
List<Fees> findByPaymentStatusNotAndStudentDepartmentId(PaymentStatus status,
Long deptId);
```

2. In FeesService.java add: List<FeesResponseDTO> getOverdue(Long departmentId); and implement: call findByPaymentStatusNot(PaymentStatus.PAID) and optionally filter by departmentId if provided.

3. In FeesController.java add:

```
@GetMapping("/overdue")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<List<FeesResponseDTO>> getOverdue(
    @RequestParam(required = false) Long departmentId) {
    return ResponseEntity.ok(feesService.getOverdue(departmentId));
}
```

> *PaymentStatus enum already exists in the model. Do not recreate it.*

Feature 6: Faculty Exam & Result Ownership

Cursor Prompt

In the College Management System Spring Boot project, add faculty ownership checks to Exam and Results mutations. Mirror the exact pattern used in AttendanceServiceImpl for faculty ownership. Steps:

1. In ExamServiceImpl.java, in the create/update methods: get currentUser from SecurityUserAccessor. If role is FACULTY, fetch the Subject linked to the exam. Check subject.getFaculty().getId().equals(currentUser.getLinkedId()). If not, throw new AccessDeniedException("You can only manage exams for your own subjects").

2. Update ExamController.java @PreAuthorize on POST/PUT/PATCH from hasRole('ADMIN') to hasAnyRole('ADMIN','FACULTY') so faculty can reach the endpoint; the ownership check in the service handles scoping.

3. Apply the exact same pattern to ResultsServiceImpl.java for create/update - faculty can only enter results for exams of subjects they teach. Check result.getExam().getSubject().getFaculty().getId() equals linkedId.

4. Update ResultsController.java POST/PUT @PreAuthorize to hasAnyRole('ADMIN','FACULTY').

5. Add GET /api/results/me to ResultsController for STUDENT role - returns results where student.id = SecurityUserAccessor.getCurrentUser().getLinkedId().

Both features fee and faculty exams
{Here’s what was implemented.

```
Fee “defaulters” / non-PAID listing

FeesRepository
```

- findByPaymentStatusNot(PaymentStatus status)
- findByPaymentStatusNotAndStudent_Department_Id(PaymentStatus status, Long departmentId) — Spring Data path for student.department.id (your findByPaymentStatusNotAndStudentDepartmentId name doesn’t map to a property path, so this derived name is required).

```
FeesService / FeesServiceImpl
```

- getOverdue(Long departmentId): if departmentId == null → findByPaymentStatusNot(PAID); else → findByPaymentStatusNotAndStudent_Department_Id(PAID, departmentId); map with existing toResponse.

```
FeesController
```

- GET /api/fees/overdue?departmentId= — @PreAuthorize("hasRole('ADMIN')"), placed before GET /{id}.

```
This lists all fee rows whose status is not PAID (e.g. PENDING, FAILED), optionally
scoped by student department.
```

```
Exam (faculty ownership)

ExamServiceImpl
```

- Injected SecurityUserAccessor.
- After loading Subject in create and update, calls assertFacultyOwnsSubjectForExam(subject) (same idea as attendance: ADMIN allowed; FACULTY needs linkedId and subject.getFaculty().getId() match; otherwise AccessDeniedException("You can only manage exams for your own subjects")).

```
ExamController
```

- POST and PUT: @PreAuthorize("hasAnyRole('ADMIN','FACULTY')").
- No PATCH on this controller. DELETE still ADMIN only.

```
Results (faculty ownership)

ResultsServiceImpl
```

- Injected SecurityUserAccessor.
- In create and update, after loading Exam, calls assertFacultyOwnsExamSubject(exam) using exam.getSubject() / subject.getFaculty().getId() vs linkedId, with AccessDeniedException("You can only manage results for your own subjects").

```
ResultsController
```

- POST and PUT: @PreAuthorize("hasAnyRole('ADMIN','FACULTY')").

```
GET /api/results/me
```

- Already implemented for STUDENT; left as-is.

```
mvn test completed successfully (exit code 0).
}
```

Feature 7: Book Return with Fine Calculation

Cursor Prompt

In the College Management System Spring Boot project, implement book return with automatic fine calculation. Steps:

1. Check if BookIssue entity has a returnDate field and a fineAmount field. If fineAmount is missing, add it as a Double field with @Column(nullable = true) - this is the ONLY allowed schema change and it is additive (nullable, so no migration needed).

2. In BookIssueService.java, add: BookIssueResponseDTO returnBook(Long issueId);

3. In BookIssueServiceImpl.java implement returnBook: fetch BookIssue by id. If already returned (returnDate != null), throw IllegalStateException. Set returnDate = LocalDate.now().

Calculate daysOverdue = ChronoUnit.DAYS.between(expectedReturnDate, returnDate) where expectedReturnDate = issueDate.plusDays(14) (14-day lending period). If daysOverdue > 0, set fineAmount = daysOverdue * 5.0 (Rs 5 per day). Save and return DTO.

4. In BookIssueController.java, add:

```
@PatchMapping("/{id}/return")
@PreAuthorize("hasAnyRole('ADMIN','FACULTY')")
public ResponseEntity<BookIssueResponseDTO> returnBook(@PathVariable Long id) {
    return ResponseEntity.ok(bookIssueService.returnBook(id));
}
```

> *Use ChronoUnit from java.time.temporal. Do not import any external library.*

{1. `BookIssue` entity

`returnDate` and `fineAmount` are already there (`Double fineAmount`), so nothing was added to the entity.

2. `BookIssueService`

- `BookIssueResponseDTO returnBook(Long issueId);`

3. `BookIssueServiceImpl.returnBook`

- Loads issue or `ResourceNotFoundException`.
- If `returnDate != null` → `IllegalStateException("Book already returned")`.
- `returnDate = LocalDate.now()`
- `expectedReturnDate = issueDate.plusDays(14)` (ignores stored `dueDate` for the fine, per your spec).
- `daysOverdue = ChronoUnit.DAYS.between(expectedReturnDate, returnDate)`
- If `daysOverdue > 0` → `fineAmount = daysOverdue * 5.0`; else `fineAmount = null`
- Sets `isReturned = true`, saves, returns `toResponse`.

4. `BookIssueController`

- `PATCH /api/book-issues/{id}/return` — `@PreAuthorize("hasAnyRole('ADMIN','FACULTY')")`, `ResponseEntity.ok(service.returnBook(id))` (field name is `service`, not `bookIssueService`).

5. `GlobalExceptionHandler`

- `IllegalStateException` → 409 CONFLICT, body uses existing `ErrorResponse` shape with error code `CONFLICT`.

`mvn test` completed successfully.

Note: Overdue is computed from issue date + 14 days, not from `dueDate`. Early return gets no fine (`fineAmount` cleared to `null`).

}

Feature 8: Dashboard Stats Endpoint

Cursor Prompt

In the College Management System Spring Boot project, create an Admin Dashboard stats endpoint. Steps:

1. Create DashboardStatsDTO.java in the dto package (no sub-package needed):

```
public class DashboardStatsDTO {
    private long totalStudents;
    private long totalFaculty;
    private long activeBooksIssued;
    private long unpaidFees;
    private long upcomingExams;
    private long upcomingEvents;
    // getters, setters or @Data
}
```

2. Create DashboardController.java in the controller package. Autowire: StudentRepository, FacultyRepository, BookIssueRepository, FeesRepository, ExamRepository, EventsRepository.

3. Add GET /api/dashboard/stats endpoint annotated @PreAuthorize("hasRole('ADMIN')"). Build DashboardStatsDTO by calling: studentRepository.count(), facultyRepository.count(), bookIssueRepository.countByReturnDateIsNull(), feesRepository.countByPaymentStatusNot(PaymentStatus.PAID), examRepository.countByExamDateAfter(LocalDate.now()), eventsRepository.countByEventDateAfter(LocalDate.now()).

4. Add any missing count query methods to the respective repositories using Spring Data JPA naming conventions (they will be auto-implemented).

> *DashboardController directly uses repositories for aggregate counts only. This is acceptable for a stats endpoint and avoids adding count methods to every service.*

{Here's what was added.

1. `DashboardStatsDTO` (`dto` package)

- Fields: `totalStudents`, `totalFaculty`, `activeBooksIssued`, `unpaidFees`, `upcomingExams`, `upcomingEvents`
- Lombok `@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`

2. `DashboardController`

- `@RequestMapping("/api/dashboard")` + constructor injection via `@RequiredArgsConstructor` (same effect as wiring the six repositories).
- `GET /api/dashboard/stats` — `@PreAuthorize("hasRole('ADMIN')")`, builds and returns `DashboardStatsDTO`.

3. Repository methods (only new methods)

Repository	Method
`BookIssueRepository`	`long countByReturnDateIsNull()`
`FeesRepository`	`long countByPaymentStatusNot(PaymentStatus status)`
`ExamRepository`	`long countByExamDateAfter(LocalDate date)`
`EventsRepository`	`long countByDateTimeAfter(LocalDateTime dateTime)`

`StudentRepository` / `FacultyRepository` use inherited `count()`.

4. Events vs your spec

`Events` has `dateTime` (`LocalDateTime`), not `eventDate`. The derived query is `countByDateTimeAfter`, and the controller uses `LocalDateTime.now()` so “upcoming” is after the current instant.

`mvn test` completed successfully.

}

Feature 9: Result Summary per Student

Cursor Prompt

In the College Management System Spring Boot project, add a result summary endpoint. Steps:

1. Create ResultSummaryDTO.java in dto/results/:

```
public class ResultSummaryDTO {
    private Long studentId;
    private String studentName;
    private double overallPercentage;
    private String overallGrade;
    private List<SubjectResult> subjectResults;
    // SubjectResult: subjectName, marksObtained, totalMarks, grade, status
}
```

2. In ResultsRepository.java add: List<Results> findByStudentId(Long studentId);

3. In ResultsService.java add: ResultSummaryDTO getSummaryByStudentId(Long studentId); and implement: fetch all results for student, map to SubjectResult list, compute overallPercentage = sum(marksObtained) / sum(totalMarks) * 100, determine overallGrade using existing Grade enum logic.

4. In ResultsController.java add:

```
@GetMapping("/student/{studentId}/summary")
@PreAuthorize("hasAnyRole('ADMIN','FACULTY')")
public ResponseEntity<ResultSummaryDTO> getSummary(@PathVariable Long
studentId) {
    return ResponseEntity.ok(resultsService.getSummaryByStudentId(studentId));
}
```

{Here's what was implemented.

1. `ResultSummaryDTO` (`dto/results/`)

- Top-level: `studentId`, `studentName`, `overallPercentage`, `overallGrade`, `subjectResults`
- Nested `SubjectResult`: `subjectName`, `marksObtained`, `totalMarks`, `grade` (String, enum name), `status` (`ResultStatus`)
- Lombok `@Data` + constructors; `subjectResults` defaults to `new ArrayList<>()`

2. `ResultsRepository`

- `findByStudentId(Long studentId)` as a `default` method that delegates to existing `findByStudent_Id` (there is no `studentId` property on `Results`, so Spring cannot derive `findByStudentId` alone).

3. `ResultsService` / `ResultsServiceImpl`

- `getSummaryByStudentId`: loads student or `ResourceNotFoundException`; loads rows via `findByStudentId`
- Per row: subject name from `exam.subject`, `totalMarks` from `exam.maxMarks` (null/≤0 excluded from overall totals)
- Overall: `overallPercentage = sum(marksObtained) / sum(maxMarks) * 100` when `sum(maxMarks) > 0`, else `0`
- Grades: there was no existing mapping in the project, so `gradeFromPercentage` maps % → `Grade` with bands: 90+ A_PLUS, 80+ A, 75+ B_PLUS, 70+ B, 65+ C_PLUS, 60+ C, 50+ D, else F; NaN → INCOMPLETE
- Row grade = same mapping on that row's % (`marksObtained` vs `maxMarks`); overallGrade from overall %

4. `ResultsController`

- `GET /api/results/student/{studentId}/summary` — `@PreAuthorize("hasAnyRole('ADMIN','FACULTY')")`, placed before `GET /{id}` so routing stays clear.

```
mvn test completed successfully.
```

}

Feature 10: Upcoming Events & Exams

Cursor Prompt

In the College Management System Spring Boot project, add upcoming events and upcoming exams endpoints. Steps:

1. In EventsRepository.java add: List<Events> findByEventDateAfterOrderByEventDateAsc(LocalDate date); (adjust field name to match your Events entity's date field exactly).

2. In ExamRepository.java add: List<Exam> findByExamDateAfterOrderByExamDateAsc(LocalDate date); (adjust field name to match Exam entity).

3. In EventsService.java add: List<EventsResponseDTO> getUpcoming(); and implement: call findByEventDateAfterOrderByEventDateAsc(LocalDate.now()).

4. In ExamService.java add: List<ExamResponseDTO> getUpcoming(); same pattern.

5. In EventsController.java add GET /api/events/upcoming (public or authenticated, your choice).
   In ExamController.java add GET /api/exams/upcoming (public or authenticated).

> *Check the actual date field name in Events.java and Exam.java before writing the repository method. Spring Data JPA method names must match entity field names exactly.*

Implementation Week Plan

Week	Features	Why This Order
Week 1	Defaulter Report, Bulk Attendance, Student /me, Faculty /me	All pure security-layer or query work. No logic dependencies between them. Highest visibility.
Week 2	Fee Overdue, Dashboard Stats, Upcoming Events/Exams	Simple query additions. Dashboard is read-only aggregation. Quick wins.
Week 3	Faculty Exam/Result Ownership, Book Return + Fine, Result Summary	These touch more business logic and need careful testing. Do last when foundation is solid.

Safety Rules for Cursor

> *Always give Cursor these rules alongside each feature prompt to prevent regressions.*

- Do not modify any existing @GetMapping, @PostMapping, @PutMapping, @DeleteMapping signatures.
- Do not remove or rename any existing service interface methods.
- Do not alter existing entity fields or JPA annotations.
- Do not change SecurityConfig.java unless explicitly told to.
- Add new @Query methods to repositories; never modify existing ones.
- If a DTO already exists, extend it rather than replacing it.
- Always run existing tests after each feature to verify nothing broke.